

Creature Trapping System - Technical Design

Project: DND35Prototype

System: Creature Trapping & Containment

Date: May 25, 2026

Status: Design Phase

SYSTEM OVERVIEW

The Creature Trapping System enables magical items to trap creatures in extradimensional spaces, store them indefinitely with complete state preservation, and release them later either as allies or enemies. This system is critical for implementing three artifact-level items: Iron Flask, Mirror of Life Trapping, and Efreeti Bottle.

Core Capabilities:

- Trap creatures with saving throw
 - Store complete creature state (HP, effects, equipment, etc.)
 - Release creatures with attitude control (friendly/hostile)
 - Support multiple trapped creatures per item (up to 15 for Mirror)
 - Persist through save/load
 - UI for viewing trapped creatures (Mirror of Life Trapping)
-

ITEMS REQUIRING THIS SYSTEM

1. Iron Flask (170,000 gp) - ★★★★★

Capacity: 1 creature

Save: Will DC 19

Range: 60 ft

Release Options:

- Serve for 1 hour
- Hostile (attacks immediately)

Special Rules:

- Can trap ANY creature type
 - Creature remains trapped even if flask destroyed
 - Opening destroyed flask releases creature
-

2. Mirror of Life Trapping (200,000 gp) - ★★★★★

Capacity: 15 creatures

Save: Will DC 23

Range: 50 ft (automatic when viewing mirror)

Release Options:

- Release individual creature (hostile)
- Release all creatures (hostile)

Special Rules:

- Automatic trap when creature views mirror
 - Can view trapped creatures by speaking name
 - **UI required** to show all trapped creatures
 - Breaking mirror releases all creatures
-

3. Efreeti Bottle (145,000 gp) - ★★★★★

Capacity: 1 creature (efreeti or outsider)

Save: Will DC 19 (for trapping outsiders)

Release Options:

- Efreeti serves for 1 hour
- Efreeti may offer 3 wishes for permanent freedom

Special Rules:

- Comes with trapped efreeti
 - Can trap outsiders (not any creature type)
 - Service mechanics more complex (wish negotiation)
-

SYSTEM ARCHITECTURE

Core Components

```

CreatureTrapSystem (manager)
├─ TrapCreature(Character, int DC, SavingThrowType)
├─ ReleaseCreature(int index, bool friendly)
├─ GetTrappedCreatures()
└─ SerializeTrappedCreatures()

TrappedCreature (data class)
├─ Name, Portrait, ID
├─ CurrentHP, MaxHP
├─ SerializedCharacterData
├─ TrapTime
└─ ActiveEffects

TrappableItem (base class for trapping items)
├─ CreatureTrapSystem TrapSystem
├─ int MaxCapacity
├─ TrapRange
└─ SaveDC

```

TRAPPED CREATURE DATA

TrappedCreature Class

```
[Serializable]
public class TrappedCreature
{
    // Basic info
    public string Name;
    public int CreatureID;
    public Sprite Portrait;

    // Current state
    public int CurrentHP;
    public int MaxHP;
    public int CurrentMana;
    public int MaxMana;

    // Serialized complete state
    public CharacterSaveData SerializedData;

    // Active effects at time of trapping
    public List<ActiveEffectData> ActiveEffects;

    // Metadata
    public DateTime TrapTime;
    public Plane PlaneOfOrigin;

    // For UI display
    public string GetStatusString()
    {
        return $"{CurrentHP}/{MaxHP} HP";
    }

    public bool IsAlive()
    {
        return CurrentHP > 0;
    }
}

[Serializable]
public class ActiveEffectData
{
    public string EffectName;
    public int Duration; // rounds remaining
    public int Magnitude;
    public string Description;
}
```

CHARACTER SERIALIZATION

Complete State Capture

```

public static class CharacterSerializer
{
    public static CharacterSaveData Serialize(Character character)
    {
        CharacterSaveData data = new CharacterSaveData();

        // Basic info
        data.Name = character.Name;
        data.Level = character.Level;
        data.Race = character.Race;
        data.Class = character.CharacterClass;
        data.Alignment = character.Alignment;

        // Ability scores
        data.Strength = character.Strength;
        data.Dexterity = character.Dexterity;
        data.Constitution = character.Constitution;
        data.Intelligence = character.Intelligence;
        data.Wisdom = character.Wisdom;
        data.Charisma = character.Charisma;

        // Current state
        data.CurrentHP = character.CurrentHP;
        data.MaxHP = character.MaxHP;
        data.CurrentMana = character.CurrentMana;
        data.MaxMana = character.MaxMana;
        data.CurrentXP = character.Experience;

        // Combat stats
        data.BaseAttackBonus = character.BaseAttackBonus;
        data.ArmorClass = character.GetTotalAC();
        data.TouchAC = character.GetTouchAC();
        data.FlatFootedAC = character.GetFlatFootedAC();

        // Saves
        data.FortitudeSave = character.GetFortitudeSave();
        data.ReflexSave = character.GetReflexSave();
        data.WillSave = character.GetWillSave();

        // Skills
        data.Skills = new Dictionary<string, int>();
        foreach (var skill in character.Skills)
        {
            data.Skills[skill.Key] = skill.Value.Ranks;
        }

        // Feats
        data.Feats = new List<string>(character.Feats);

        // Spells known/prepared
        data.SpellsKnown = new List<string>(character.SpellsKnown);
        data.SpellsPrepared = new List<string>(character.SpellsPrepared);
        data.SpellSlotsRemaining = new Dictionary<int, int>(character.SpellSlotsRemaining);

        // Equipment (serialize items)
        data.EquippedItems = new List<ItemSaveData>();
        foreach (var item in character.EquippedItems)
        {
            data.EquippedItems.Add(ItemSerializer.Serialize(item));
        }
    }
}

```

```

data.Inventory = new List<ItemSaveData>();
foreach (var item in character.Inventory)
{
    data.Inventory.Add(ItemSerializer.Serialize(item));
}

// Active effects (buffs, debuffs, etc.)
data.ActiveEffects = new List<ActiveEffectData>();
foreach (var effect in character.ActiveEffects)
{
    data.ActiveEffects.Add(new ActiveEffectData
    {
        EffectName = effect.Name,
        Duration = effect.RemainingDuration,
        Magnitude = effect.Magnitude,
        Description = effect.Description
    });
}

// Position and scene (for restoration)
data.LastPosition = character.transform.position;
data.LastRotation = character.transform.rotation;
data.LastSceneName = SceneManager.GetActiveScene().name;

return data;
}

public static Character Deserialize(CharacterSaveData data)
{
    // Create new character instance
    GameObject charObj = Instantiate(Resources.Load<GameObject>("Prefabs/Character"));
    Character character = charObj.GetComponent<Character>();

    // Restore basic info
    character.Name = data.Name;
    character.Level = data.Level;
    character.Race = data.Race;
    character.CharacterClass = data.Class;
    character.Alignment = data.Alignment;

    // Restore abilities
    character.Strength = data.Strength;
    character.Dexterity = data.Dexterity;
    character.Constitution = data.Constitution;
    character.Intelligence = data.Intelligence;
    character.Wisdom = data.Wisdom;
    character.Charisma = data.Charisma;

    // Restore current state
    character.CurrentHP = data.CurrentHP;
    character.MaxHP = data.MaxHP;
    character.CurrentMana = data.CurrentMana;
    character.MaxMana = data.MaxMana;
    character.Experience = data.CurrentXP;

    // Restore combat stats
    character.BaseAttackBonus = data.BaseAttackBonus;

    // Restore saves
    character.FortitudeSaveBase = data.FortitudeSave;
    character.ReflexSaveBase = data.ReflexSave;
    character.WillSaveBase = data.WillSave;

```

```

    // Restore skills
    foreach (var skillData in data.Skills)
    {
        character.SetSkillRanks(skillData.Key, skillData.Value);
    }

    // Restore feats
    character.Feats = new List<string>(data.Feats);

    // Restore spells
    character.SpellsKnown = new List<string>(data.SpellsKnown);
    character.SpellsPrepared = new List<string>(data.SpellsPrepared);
    character.SpellSlotsRemaining = new Dictionary<int, int>(data.SpellSlotsRemain
ing);

    // Restore equipment
    foreach (var itemData in data.EquippedItems)
    {
        Item item = ItemSerializer.Deserialize(itemData);
        character.EquipItem(item);
    }

    foreach (var itemData in data.Inventory)
    {
        Item item = ItemSerializer.Deserialize(itemData);
        character.AddToInventory(item);
    }

    // Restore active effects
    foreach (var effectData in data.ActiveEffects)
    {
        ActiveEffect effect = new ActiveEffect
        {
            Name = effectData.EffectName,
            RemainingDuration = effectData.Duration,
            Magnitude = effectData.Magnitude,
            Description = effectData.Description
        };
        character.AddEffect(effect);
    }

    // Position (will be set when creature is released)
    character.transform.position = data.LastPosition;
    character.transform.rotation = data.LastRotation;

    return character;
}
}

```

CREATURE TRAP SYSTEM

CreatureTrapSystem.cs


```

public class CreatureTrapSystem
{
    public List<TrappedCreature> TrappedCreatures;
    public int MaxCapacity;
    public WondrousItem OwnerItem; // Item that owns this trap system

    public CreatureTrapSystem(int capacity, WondrousItem owner)
    {
        MaxCapacity = capacity;
        TrappedCreatures = new List<TrappedCreature>();
        OwnerItem = owner;
    }

    /// <summary>
    /// Attempt to trap a creature
    /// </summary>
    public bool TrapCreature(Character target, int saveDC, SavingThrowType saveType =
    SavingThrowType.Will)
    {
        // Check capacity
        if (TrappedCreatures.Count >= MaxCapacity)
        {
            CombatLog.Add($"{OwnerItem.Name} is full! Cannot trap {target.Name}.");
            return false;
        }

        // Check if already trapped somewhere
        if (target.IsTrapped)
        {
            CombatLog.Add($"{target.Name} is already trapped!");
            return false;
        }

        // Saving throw
        bool saved = false;
        switch (saveType)
        {
            case SavingThrowType.Will:
                saved = SavingThrows.MakeWillSave(target, saveDC);
                break;
            case SavingThrowType.Reflex:
                saved = SavingThrows.MakeReflexSave(target, saveDC);
                break;
            case SavingThrowType.Fortitude:
                saved = SavingThrows.MakeFortSave(target, saveDC);
                break;
        }

        if (saved)
        {
            CombatLog.Add($"{target.Name} resists being trapped! ({saveType} save DC
            {saveDC})");
            return false;
        }

        // Trap the creature
        TrappedCreature trapped = new TrappedCreature
        {
            Name = target.Name,
            CreatureID = target.ID,
            Portrait = target.Portrait,
            CurrentHP = target.CurrentHP,

```

```

        MaxHP = target.MaxHP,
        CurrentMana = target.CurrentMana,
        MaxMana = target.MaxMana,
        SerializedData = CharacterSerializer.Serialize(target),
        TrapTime = System.DateTime.Now,
        PlaneOfOrigin = PlanarTravelSystem.GetCurrentPlane(target)
    };

    // Copy active effects
    trapped.ActiveEffects = new List<ActiveEffectData>();
    foreach (var effect in target.ActiveEffects)
    {
        trapped.ActiveEffects.Add(new ActiveEffectData
        {
            EffectName = effect.Name,
            Duration = effect.RemainingDuration,
            Magnitude = effect.Magnitude,
            Description = effect.Description
        });
    }

    TrappedCreatures.Add(trapped);

    // Remove creature from game world
    RemoveCreatureFromWorld(target);

    CombatLog.Add($"{target.Name} is trapped in {OwnerItem.Name}!");

    return true;
}

/// <summary>
/// Release a trapped creature
/// </summary>
public Character ReleaseCreature(int index, bool friendlyToOwner, Vector3 release-
Position)
{
    if (index < 0 || index >= TrappedCreatures.Count)
    {
        Debug.LogError($"Invalid trap index: {index}");
        return null;
    }

    TrappedCreature trapped = TrappedCreatures[index];

    // Check if creature is still alive (could be killed while trapped in some
    edge cases)
    if (!trapped.IsAlive())
    {
        CombatLog.Add($"{trapped.Name} is dead and cannot be released.");
        TrappedCreatures.RemoveAt(index);
        return null;
    }

    // Deserialize and restore creature
    Character released = CharacterSerializer.Deserialize(trapped.SerializedData);

    // Set position
    released.transform.position = releasePosition;

    // Set attitude
    if (friendlyToOwner)
    {

```

```

        released.Attitude = Attitude.Friendly;
        released.Owner = OwnerItem.Owner;
        CombatLog.Add($"{released.Name} is released and serves you!");
    }
    else
    {
        released.Attitude = Attitude.Hostile;
        released.Target = OwnerItem.Owner; // Attack the person who released them
        CombatLog.Add($"{released.Name} is released and attacks!");
    }

    // Remove from trap
    TrappedCreatures.RemoveAt(index);

    // Add to current scene
    AddCreatureToWorld(released);

    return released;
}

/// <summary>
/// Release all trapped creatures
/// </summary>
public List<Character> ReleaseAll(bool friendlyToOwner, Vector3 releasePosition)
{
    List<Character> released = new List<Character>();

    // Release in reverse order to avoid index issues
    for (int i = TrappedCreatures.Count - 1; i >= 0; i--)
    {
        Character creature = ReleaseCreature(i, friendlyToOwner, releasePosition);
        if (creature != null)
        {
            released.Add(creature);
        }
    }

    CombatLog.Add($"{OwnerItem.Name} releases all trapped creatures!");

    return released;
}

/// <summary>
/// Get list of trapped creatures (for UI)
/// </summary>
public List<TrappedCreature> GetTrappedCreatures()
{
    return new List<TrappedCreature>(TrappedCreatures);
}

/// <summary>
/// Check if trap is full
/// </summary>
public bool IsFull()
{
    return TrappedCreatures.Count >= MaxCapacity;
}

/// <summary>
/// Get number of trapped creatures
/// </summary>
public int GetCount()
{

```

```

        return TrappedCreatures.Count;
    }

    /// <summary>
    /// Find trapped creature by name
    /// </summary>
    public int FindCreatureByName(string name)
    {
        for (int i = 0; i < TrappedCreatures.Count; i++)
        {
            if (TrappedCreatures[i].Name.Equals(name, System.StringComparison.OrdinalIgnoreCase))
            {
                return i;
            }
        }
        return -1;
    }

    /// <summary>
    /// Serialize for save/load
    /// </summary>
    public string SerializeToJson()
    {
        return JsonUtility.ToJson(new TrapSystemSaveData
        {
            TrappedCreatures = TrappedCreatures,
            MaxCapacity = MaxCapacity
        });
    }

    /// <summary>
    /// Deserialize from save
    /// </summary>
    public void DeserializeFromJson(string json)
    {
        TrapSystemSaveData data = JsonUtility.FromJson<TrapSystemSaveData>(json);
        TrappedCreatures = data.TrappedCreatures;
        MaxCapacity = data.MaxCapacity;
    }

    // Helper methods
    private void RemoveCreatureFromWorld(Character creature)
    {
        creature.IsTrapped = true;
        creature.gameObject.SetActive(false);

        // Remove from combat if in combat
        if (CombatManager.IsInCombat && CombatManager.Combatants.Contains(creature))
        {
            CombatManager.RemoveCombatant(creature);
        }
    }

    private void AddCreatureToWorld(Character creature)
    {
        creature.IsTrapped = false;
        creature.gameObject.SetActive(true);

        // Add to combat if owner is in combat
        if (CombatManager.IsInCombat && OwnerItem.Owner != null)
        {
            CombatManager.AddCombatant(creature);
        }
    }

```

```
    }  
  }  
}  
  
[Serializable]  
public class TrapSystemSaveData  
{  
    public List<TrappedCreature> TrappedCreatures;  
    public int MaxCapacity;  
}
```

SERVICE MECHANICS

For items like Iron Flask and Efreeti Bottle, creatures can serve the owner for a limited time.

ServiceController.cs

```

public class ServiceController : MonoBehaviour
{
    public Character ServingCreature;
    public Character Master;
    public float ServiceDurationRounds;
    public float RoundsRemaining;

    void Update()
    {
        if (RoundsRemaining > 0)
        {
            RoundsRemaining -= Time.deltaTime / 6.0f; // Convert seconds to rounds

            if (RoundsRemaining <= 0)
            {
                EndService();
            }
        }
    }

    public void StartService(Character creature, Character master, int durationRounds)
    {
        ServingCreature = creature;
        Master = master;
        ServiceDurationRounds = durationRounds;
        RoundsRemaining = durationRounds;

        creature.Attitude = Attitude.Friendly;
        creature.Owner = master;

        CombatLog.Add($"{creature.Name} will serve {master.Name} for {durationRounds}
rounds (1 hour).");
    }

    private void EndService()
    {
        CombatLog.Add($"{ServingCreature.Name}'s service ends. They are no longer
bound!");

        // Creature becomes neutral or returns to trap
        ServingCreature.Attitude = Attitude.Neutral;
        ServingCreature.Owner = null;

        // Optionally: creature disappears/returns to home plane
        Destroy(ServingCreature.gameObject);
    }

    public void GiveCommand(string command)
    {
        // Creature obeys commands during service
        CombatLog.Add($"{Master.Name} commands {ServingCreature.Name}: {command}");

        // AI executes command
        ServingCreature.ExecuteCommand(command);
    }
}

```

ITEM IMPLEMENTATIONS

Iron Flask

```

public class IronFlask : WondrousItem
{
    public CreatureTrapSystem TrapSystem;
    public int TrapRange = 60; // feet
    public int SaveDC = 19;

    public IronFlask()
    {
        Name = "Iron Flask";
        Price = 170000;
        Slot = EquipmentSlot.Slotless;
        CasterLevel = 20;

        TrapSystem = new CreatureTrapSystem(capacity: 1, owner: this);
    }

    public bool TrapCreature(Character target)
    {
        // Check range
        float distance = Vector3.Distance(Owner.transform.position, target.transform.position);
        if (distance > TrapRange)
        {
            CombatLog.Add($"{target.Name} is out of range! (Max {TrapRange} ft)");
            return false;
        }

        // Attempt trap (Will DC 19)
        return TrapSystem.TrapCreature(target, SaveDC, SavingThrowType.Will);
    }

    public Character ReleaseCreature(bool serve = false)
    {
        if (TrapSystem.GetCount() == 0)
        {
            CombatLog.Add("The Iron Flask is empty!");
            return null;
        }

        Vector3 releasePos = Owner.transform.position + Owner.transform.forward * 5;

        if (serve)
        {
            // Creature serves for 1 hour (600 rounds)
            Character released = TrapSystem.ReleaseCreature(0, friendlyToOwner: true, releasePos);

            if (released != null)
            {
                ServiceController service = released.gameObject.AddComponent<ServiceController>();
                service.StartService(released, Owner, durationRounds: 600);
            }

            return released;
        }
        else
        {
            // Creature is hostile
            return TrapSystem.ReleaseCreature(0, friendlyToOwner: false, releasePos);
        }
    }
}

```



```
public override string GetDescription()
{
    int count = TrapSystem.GetCount();
    if (count == 0)
    {
        return "An empty brass flask with a lead stopper. It can trap one creature.";
    }
    else
    {
        string creatureName = TrapSystem.GetTrappedCreatures()[0].Name;
        return $"An brass flask containing {creatureName}. The creature can be released to serve you or freed.";
    }
}
```

Mirror of Life Trapping

```

public class MirrorOfLifeTrapping : WondrousItem
{
    public CreatureTrapSystem TrapSystem;
    public int TrapRange = 50; // feet
    public int SaveDC = 23;
    public bool IsActive = true;

    private TrappedCreaturesUI ui;

    public MirrorOfLifeTrapping()
    {
        Name = "Mirror of Life Trapping";
        Price = 200000;
        Slot = EquipmentSlot.Slotless;
        CasterLevel = 17;

        TrapSystem = new CreatureTrapSystem(capacity: 15, owner: this);
    }

    void Update()
    {
        if (!IsActive)
            return;

        // Check for creatures looking at mirror
        List<Character> nearbyCreatures = GetNearbyCreatures(TrapRange);

        foreach (var creature in nearbyCreatures)
        {
            if (IsLookingAtMirror(creature) && !creature.IsTrapped)
            {
                AttemptAutoTrap(creature);
            }
        }
    }

    private void AttemptAutoTrap(Character viewer)
    {
        if (TrapSystem.IsFull())
        {
            CombatLog.Add($"The mirror cannot trap {viewer.Name} - it is full!");
            return;
        }

        // Automatic trap (Will DC 23)
        TrapSystem.TrapCreature(viewer, SaveDC, SavingThrowType.Will);
    }

    private bool IsLookingAtMirror(Character creature)
    {
        // Raycast from creature to mirror
        Vector3 direction = (transform.position - creature.transform.position).normalized;

        float angle = Vector3.Angle(creature.transform.forward, direction);

        // Creature is looking at mirror if within 60 degrees
        return angle < 60.0f;
    }

    /// <summary>
    /// View trapped creatures (command word)
    /// </summary>

```

```

public void ViewTrappedCreatures()
{
    if (ui == null)
    {
        ui = UIManager.ShowPanel<TrappedCreaturesUI>();
        ui.Initialize(this);
    }

    ui.UpdateList(TrapSystem.GetTrappedCreatures());
    ui.Show();
}

/// <summary>
/// View specific creature by name (command word + name)
/// </summary>
public void ViewCreature(string name)
{
    int index = TrapSystem.FindCreatureByName(name);

    if (index >= 0)
    {
        TrappedCreature creature = TrapSystem.GetTrappedCreatures()[index];

        // Display creature in mirror
        MirrorDisplay.ShowCreature(creature);

        CombatLog.Add($"You see {creature.Name} trapped in the mirror.
({creature.GetStatusString()})");
    }
    else
    {
        CombatLog.Add($"No creature named '{name}' is trapped in the mirror.");
    }
}

/// <summary>
/// Release single creature
/// </summary>
public Character ReleaseCreature(int index)
{
    Vector3 releasePos = transform.position + transform.forward * 5;
    return TrapSystem.ReleaseCreature(index, friendlyToOwner: false, releasePos);
}

/// <summary>
/// Release all creatures (e.g., when mirror breaks)
/// </summary>
public List<Character> ReleaseAll()
{
    Vector3 releasePos = transform.position;
    return TrapSystem.ReleaseAll(friendlyToOwner: false, releasePos);
}

/// <summary>
/// Mirror destroyed - release all
/// </summary>
public override void OnDestroyed()
{
    CombatLog.Add("The Mirror of Life Trapping shatters! All trapped creatures are re-
leased!");
    ReleaseAll();
}

```

```
} }
```

Efreeti Bottle

```

public class EfreetiBottle : WondrousItem
{
    public CreatureTrapSystem TrapSystem;
    public int SaveDC = 19;
    public bool ContainsEfreeti = true;

    public EfreetiBottle()
    {
        Name = "Efreeti Bottle";
        Price = 145000;
        Slot = EquipmentSlot.Slotless;
        CasterLevel = 14;

        TrapSystem = new CreatureTrapSystem(capacity: 1, owner: this);

        // Bottle starts with trapped efreeti
        if (ContainsEfreeti)
        {
            TrapStartingEfreeti();
        }
    }

    private void TrapStartingEfreeti()
    {
        // Create efreeti
        Character efreeti = CreatureDatabase.CreateCreature("Efreeti");

        // Trap without save (pre-trapped)
        TrappedCreature trapped = new TrappedCreature
        {
            Name = efreeti.Name,
            CreatureID = efreeti.ID,
            Portrait = efreeti.Portrait,
            CurrentHP = efreeti.MaxHP,
            MaxHP = efreeti.MaxHP,
            SerializedData = CharacterSerializer.Serialize(efreeti),
            TrapTime = System.DateTime.Now
        };

        TrapSystem.TrappedCreatures.Add(trapped);
        Destroy(efreeti.gameObject);
    }

    public Character ReleaseEfreeti()
    {
        if (TrapSystem.GetCount() == 0)
        {
            CombatLog.Add("The Efreeti Bottle is empty!");
            return null;
        }

        Vector3 releasePos = Owner.transform.position + Owner.transform.forward * 5;
        Character efreeti = TrapSystem.ReleaseCreature(0, friendlyToOwner: true, releasePos);

        if (efreeti != null)
        {
            // Efreeti serves for 1 hour
            ServiceController service = efreeti.gameObject.AddComponent<ServiceController>();
            service.StartService(efreeti, Owner, durationRounds: 600);
        }
    }
}

```

```

        // 10% chance efreeti offers wishes for freedom
        if (Random.Range(0, 100) < 10)
        {
            OfferWishesForFreedom(efreeti);
        }
    }

    return efreeti;
}

private void OfferWishesForFreedom(Character efreeti)
{
    CombatLog.Add($"{efreeti.Name} offers you three wishes in exchange for
permanent freedom!");

    // Show dialogue UI
    DialogueManager.ShowDialogue(new Dialogue
    {
        Speaker = efreeti,
        Text = "I offer you three wishes, mortal, if you grant me permanent
freedom from this accursed bottle!",
        Options = new List<DialogueOption>
        {
            new DialogueOption
            {
                Text = "Accept the offer (3 wishes, efreeti goes free)",
                Action = () => AcceptWishOffer(efreeti)
            },
            new DialogueOption
            {
                Text = "Refuse (efreeti serves for 1 hour, then returns to
bottle)",
                Action = () => RefuseWishOffer(efreeti)
            }
        }
    });
}

private void AcceptWishOffer(Character efreeti)
{
    // Grant 3 wishes
    WishManager.GrantWishes(Owner, 3);

    // Efreeti is permanently freed
    efreeti.Owner = null;
    efreeti.Attitude = Attitude.Neutral;

    CombatLog.Add($"{efreeti.Name} is freed! You now have 3 wishes.");

    // Bottle becomes empty permanently
    ContainsEfreeti = false;
}

private void RefuseWishOffer(Character efreeti)
{
    CombatLog.Add($"{efreeti.Name} grudgingly continues to serve.");
}

/// <summary>
/// Trap an outsider
/// </summary>
public bool TrapOutsider(Character target)
{

```



```
// Can only trap outsiders
if (target.Type != CreatureType.Outsider)
{
    CombatLog.Add($"The Efreeti Bottle can only trap outsiders! {target.Name}
is {target.Type}.");
    return false;
}

// Will save DC 19
return TrapSystem.TrapCreature(target, SaveDC, SavingThrowType.Will);
}
}
```

UI FOR TRAPPED CREATURES

TrappedCreaturesUI.cs

```

public class TrappedCreaturesUI : MonoBehaviour
{
    public GameObject RowPrefab;
    public Transform ContentParent;
    public Button ReleaseAllButton;
    public Button CloseButton;

    private MirrorOfLifeTrapping mirror;
    private List<TrappedCreatureRow> rows = new List<TrappedCreatureRow>();

    public void Initialize(MirrorOfLifeTrapping mirrorItem)
    {
        mirror = mirrorItem;

        ReleaseAllButton.onClick.AddListener(OnReleaseAllClicked);
        CloseButton.onClick.AddListener(OnCloseClicked);
    }

    public void UpdateList(List<TrappedCreature> creatures)
    {
        // Clear existing rows
        foreach (var row in rows)
        {
            Destroy(row.gameObject);
        }
        rows.Clear();

        // Create row for each trapped creature
        for (int i = 0; i < creatures.Count; i++)
        {
            TrappedCreature creature = creatures[i];

            GameObject rowObj = Instantiate(RowPrefab, ContentParent);
            TrappedCreatureRow row = rowObj.GetComponent<TrappedCreatureRow>();

            row.SetData(
                creature.Name,
                creature.Portrait,
                creature.GetStatusString(),
                i
            );

            int index = i; // Capture for closure
            row.OnReleaseClicked += () => OnReleaseCreatureClicked(index);

            rows.Add(row);
        }

        // Update UI state
        ReleaseAllButton.interactable = creatures.Count > 0;
    }

    private void OnReleaseCreatureClicked(int index)
    {
        Character released = mirror.ReleaseCreature(index);

        if (released != null)
        {
            // Refresh list
            UpdateList(mirror.TrapSystem.GetTrappedCreatures());
        }
    }
}

```

```

    private void OnReleaseAllClicked()
    {
        mirror.ReleaseAll();
        UpdateList(mirror.TrapSystem.GetTrappedCreatures());
    }

    private void OnCloseClicked()
    {
        Hide();
    }

    public void Show()
    {
        gameObject.SetActive(true);
    }

    public void Hide()
    {
        gameObject.SetActive(false);
    }
}

public class TrappedCreatureRow : MonoBehaviour
{
    public Image Portrait;
    public Text NameText;
    public Text StatusText;
    public Button ReleaseButton;

    public event System.Action OnReleaseClicked;

    private int index;

    void Start()
    {
        ReleaseButton.onClick.AddListener(() => OnReleaseClicked?.Invoke());
    }

    public void SetData(string name, Sprite portrait, string status, int creatureIndex)
    {
        NameText.text = name;
        StatusText.text = status;
        Portrait.sprite = portrait;
        index = creatureIndex;
    }
}

```

SAVE/LOAD INTEGRATION

```

public static class CreatureTrapSaveLoad
{
    public static void SaveTrappingItems(SaveData saveData)
    {
        saveData.TrappedCreatureData = new List<TrappedItemSaveData>();

        // Find all items with trap systems
        IronFlask[] flasks = FindObjectsOfType<IronFlask>();
        foreach (var flask in flasks)
        {
            saveData.TrappedCreatureData.Add(new TrappedItemSaveData
            {
                ItemID = flask.ID,
                ItemType = "IronFlask",
                TrapSystemData = flask.TrapSystem.SerializeToJson()
            });
        }

        MirrorOfLifeTrapping[] mirrors = FindObjectsOfType<MirrorOfLifeTrapping>();
        foreach (var mirror in mirrors)
        {
            saveData.TrappedCreatureData.Add(new TrappedItemSaveData
            {
                ItemID = mirror.ID,
                ItemType = "MirrorOfLifeTrapping",
                TrapSystemData = mirror.TrapSystem.SerializeToJson()
            });
        }

        EfreetiBottle[] bottles = FindObjectsOfType<EfreetiBottle>();
        foreach (var bottle in bottles)
        {
            saveData.TrappedCreatureData.Add(new TrappedItemSaveData
            {
                ItemID = bottle.ID,
                ItemType = "EfreetiBottle",
                TrapSystemData = bottle.TrapSystem.SerializeToJson()
            });
        }
    }

    public static void LoadTrappingItems(SaveData saveData)
    {
        foreach (var itemData in saveData.TrappedCreatureData)
        {
            WondrousItem item = FindItemByID(itemData.ItemID);

            if (item == null)
                continue;

            switch (itemData.ItemType)
            {
                case "IronFlask":
                    IronFlask flask = item as IronFlask;
                    flask.TrapSystem.DeserializeFromJson(itemData.TrapSystemData);
                    break;

                case "MirrorOfLifeTrapping":
                    MirrorOfLifeTrapping mirror = item as MirrorOfLifeTrapping;
                    mirror.TrapSystem.DeserializeFromJson(itemData.TrapSystemData);
                    break;
            }
        }
    }
}

```

```

        case "EfreetiBottle":
            EfreetiBottle bottle = item as EfreetiBottle;
            bottle.TrapSystem.DeserializeFromJson(itemData.TrapSystemData);
            break;
        }
    }
}

[Serializable]
public class TrappedItemSaveData
{
    public int ItemID;
    public string ItemType;
    public string TrapSystemData; // JSON
}

```

TESTING CHECKLIST

Basic Trapping

- [] Iron Flask traps creature with Will save
- [] Mirror traps creature when viewed
- [] Efreeti Bottle traps outsiders
- [] Creatures fail/succeed saves correctly
- [] Cannot trap beyond capacity
- [] Trapped creature removed from world

State Preservation

- [] Creature HP preserved when trapped
- [] Active effects preserved
- [] Equipment preserved
- [] Spells/abilities preserved
- [] All stats match on release

Release Mechanics

- [] Iron Flask: friendly service for 1 hour
- [] Iron Flask: hostile release attacks
- [] Mirror: release is always hostile
- [] Mirror: release all works
- [] Efreeti: service for 1 hour
- [] Efreeti: wish negotiation triggers

UI (Mirror)

- [] List shows all trapped creatures
- [] Portraits display correctly
- [] HP/status accurate
- [] Individual release works
- [] Release all works

- [] UI updates after release

Save/Load

- [] Trapped creatures persist through save/load
- [] All creature data restored correctly
- [] Items maintain trapped creatures after load
- [] No data corruption

Edge Cases

- [] Trapping already-trapped creature fails
- [] Flask destroyed: creature remains trapped
- [] Mirror destroyed: all creatures released
- [] Service expires: creature freed/vanishes
- [] Creature dies while trapped (if possible)

PERFORMANCE CONSIDERATIONS

Potential Issues:

1. **Serialization overhead** for complex characters
2. **Mirror auto-trap** checking every frame
3. **UI updates** for 15 creatures

Optimizations:

- Cache serialized data (only serialize once)
 - Mirror: use trigger collider + raycast (not Update loop for all creatures)
 - UI: lazy load portraits, virtual scrolling for long lists
 - Limit simultaneous active trap items (e.g., max 3 mirrors in scene)
-

ESTIMATED EFFORT

Implementation Time: 2-3 weeks

Breakdown:

- Character serialization system: 3 days
- CreatureTrapSystem core: 3 days
- Iron Flask implementation: 1 day
- Mirror of Life Trapping + UI: 4 days
- Efrete Bottle + service mechanics: 3 days
- Save/load integration: 2 days
- Testing + bug fixes: 3 days

Total: 19 days (~3 weeks)

Document Version: 1.0

Last Updated: May 25, 2026

Status: Ready for Implementation